databricks

Advance Lakeflow Jobs Features

LECTURE

Best Practices

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's establish the comprehensive best practices that ensure production success across all aspects of Lakeflow Jobs implementation.

Best Practices

Compute & Cost Optimization

- Use job or **serverless clusters** in production
- Avoid interactive clusters for production workloads
- **Enable Photon** for faster and cheaper execution
- **Reuse clusters** to reduce startup time and cost

Orchestration & Modularity

- **Break complex pipelines** into modular jobs/run jobs
- Use **multi-task jobs** for parallel and scalable execution
- Apply conditional logic using Run If, If/Else, For Each task
- Limit jobs to **manageable task counts** for maintainability (Limited to 1000 task)

Monitoring & Governance

- **Use service principals** for job ownership and data access
- Configure alerts for **failures, delays, and completions**
- Leverage **Repair & Run** for reducing cost on re-processing
- Parameterize tasks for **reusability and flexibility**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Compute & Cost Optimization Practices:

- Production Compute: Always use job clusters or serverless compute in production environments to ensure cost efficiency and resource isolation
- Interactive Cluster Avoidance: Reserve interactive clusters strictly for development and ad-hoc analysis to prevent production cost overruns and resource conflicts
- Photon Enablement: Enable Photon acceleration across all eligible workloads for significant performance improvements and cost reductions
- Cluster Reuse: Design workflows to reuse clusters across tasks when possible, reducing startup overhead and improving cost efficiency

Best Practices

Compute & Cost Optimization

- Use job or **serverless clusters** in production
- Avoid interactive clusters for production workloads
- **Enable Photon** for faster and cheaper execution
- **Reuse clusters** to reduce startup time and cost

Orchestration & Modularity

- **Break complex pipelines** into modular jobs/run jobs
- Use **multi-task jobs** for parallel and scalable execution
- Apply conditional logic using Run If, If/Else, For Each task
- Limit jobs to **manageable task counts** for maintainability (Limited to 1000 task)

Monitoring & Governance

- **Use service principals** for job ownership and data access
- Configure alerts for **failures, delays, and completions**
- Leverage **Repair & Run** for reducing cost on re-processing
- Parameterize tasks for **reusability and flexibility**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Orchestration & Modularity Practices:

- Modular Architecture: Break complex pipelines into logical, maintainable modules using the run job task pattern for better maintainability and team collaboration
- Multi-Task Design: Leverage multi-task jobs for parallel execution and improved resource utilization while maintaining workflow clarity
- Advanced Logic: Implement sophisticated business logic using Run If, If/Else, and For Each tasks to handle complex real-world scenarios
- Task Limitations: Keep individual jobs under 1000 tasks to maintain manageability and performance, using modular design to handle larger workflows

Best Practices

Compute & Cost Optimization

- Use job or **serverless clusters** in production
- Avoid interactive clusters for production workloads
- **Enable Photon** for faster and cheaper execution
- **Reuse clusters** to reduce startup time and cost

Orchestration & Modularity

- **Break complex pipelines** into modular jobs/run jobs
- Use **multi-task jobs** for parallel and scalable execution
- Apply conditional logic using Run If, If/Else, For Each task
- Limit jobs to **manageable task counts** for maintainability (Limited to 1000 task)

Monitoring & Governance

- **Use service principals** for job ownership and data access
- Configure alerts for **failures, delays, and completions**
- Leverage **Repair & Run** for reducing cost on re-processing
- Parameterize tasks for **reusability and flexibility**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Monitoring & Governance Practices:

- Service Principal Usage: Use service principals rather than personal accounts for job ownership and data access to ensure continuity and proper access control
- Comprehensive Alerting: Configure alerts for failures, delays, and completions with appropriate escalation and notification strategies
- Recovery Optimization: Leverage Repair & Run capabilities to minimize costs and recovery time when handling failures
- Parameterization: Design tasks with proper parameterization for maximum reusability and flexibility across environments and use cases

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.